

Ungraded File IO Self-Exercises

In these self-exercises for File IO, there are four programming practices. It is suggested to complete these exercises on your own, as practice on the topic, File IO, which is included in the syllabus.

Self-exercise 1: Reading a File

Problem description:

A very long file of numbers is given as numbers.txt of 300 lines. There are 2 integers x, y on every line. We would like to print the lines requested by the user, from start to end inclusively. The file starts on line 1.

In every line, the first integer x tells how many times to print the second integer y in the console output (stdin), e.g., 3 10 in the file would give an output as, 10 10 10.

Sample input 1:

```
1 2
```

Sample output 1:

```
75 75 75
32 32 32 32 32 32
```

Sample input 2:

```
100 105
```

Sample output 2:

```
11 11 11 11 11 11 11 11 11
97 97
32 32 32 32 32 32 32 32 32 32
29 29 29 29 29 29 29 29 29 29
7 7 7 7 7 7 7 7 7
92 92
```

Sample input 3:

```
300 300
```

Sample output 3:

```
92 92 92 92 92 92 92
```

Self-exercise 2: `fgets()` and `fscanf()`

Problem description:

In the lecture notes we have covered two ways to get a string from user - ``fgets()`` and ``fscanf()``. Do you know the difference between these two methods? In this exercise, let's try them out!

- Create two char array `inputArray1` and `inputArray2` of size 20 and set all elements to 0
- Open the file `data.txt`
- Use `fgets()` to get a string from user into `inputArray1`
- Print the 20 elements of `inputArray1` one by one, as a number (i.e. ASCII code)
- Print `inputArray1` to the screen, as a string
- Use `fscanf()` to get a string from user into `inputArray2`
- Print the 20 elements of `inputArray2` one by one, as a number (i.e. ASCII code)
- Print `inputArray2` to the screen, as a string
- Close the file

Sample input 1 – in `data.txt`:

```
HELLO WORLD  
HELLO WORLD
```

Sample output 1 – stdout:

```
72 69 76 76 79 32 87 79 82 76 68 10 0 0 0 0 0 0 0 0  
HELLO WORLD  
72 69 76 76 79 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0  
HELLO
```

Sample input 2 – in `data.txt`:

```
HELLO-WORLD  
HELLO-WORLD
```

Sample output 2 – stdout:

```
72 69 76 76 79 45 87 79 82 76 68 10 0 0 0 0 0 0 0 0  
HELLO-WORLD  
72 69 76 76 79 45 87 79 82 76 68 0 0 0 0 0 0 0 0 0  
HELLO-WORLD
```

Sample input 3 – in `data.txt`:

```
I want Christmas  
I want Christmas
```

Sample output 3 – stdout:

```
73 32 119 97 110 116 32 67 104 114 105 115 116 109 97 115 10 0 0 0  
I want Christmas  
73 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0  
I
```

Self-exercise 3: Reverse each line

Problem description:

Write a C program that reverse the content of each line of input.txt, and save the results to output.txt.

- Open input.txt using read mode, open output.txt using write mode.
- Read a line from input.txt to a character array `str`.
- If the last character of `str` is newline character, change it to 0.
- Use a loop to print the content of `str` from the end to the front to the output.txt.
- Repeat the previous 3 steps until reaching the EOF.
- Close both files.

You can assume that one line does not contain more than 100 characters.

Sample input 1 – in input.txt:

```
1234567890
Hello
```

Sample output 1 – in output.txt:

```
0987654321
olleH
```

Sample input 2 – in input.txt:

```
ENGG1110 is fun.
I love writing programming.
BYE ~ ~ ~
```

Sample output 2 – in output.txt:

```
.nuf si 0111GGNE
.gnimmargorp gnitirw evol I
~ ~ ~ EYB
```

Self-exercise 4: Variance

Problem description:

In this practice, you are going to calculate variance of a list of number without using array! Is this amazing? The key idea is $\text{Var}[X] = E[X^2] - E[X]^2$.

- Open the file, value.txt, using read mode.
- Initialize `sum_of_square = 0`, `sum = 0`, `N = 0`.
- Repeat the following steps, until no more numbers can be read from the file:
 - Read a number from the file.
 - Add the number to `sum`.
 - Add the number $\times$ number to `sum_of_square`.
 - Increase `N` by 1
- Calculate `variance = sum_of_square / N - (sum / N)2`.
- Output variance up to 4 decimal places.
- Close the file.

Sample input 1 – in value.txt:

```
1 2 3 4 5 6 7 8 9 10
```

Sample output 1 – stdout:

```
8.2500
```

Sample input 2 – in value.txt:

```
2 2 2 2
3 3
4 4 4 4
```

Sample output 2 – stdout:

```
0.8000
```

Sample input 3 – in value.txt:

```
10    10    10
26    26    99    99
123   1     1           1
9     0     9     0
```

Sample output 3 – stdout:

```
1637.5289
```